```
#!/usr/bin/env python3
"""
Universal JSONL Manifest & Marker Generator
Scans the vault, extracts metadata/headers, calculates hashes, and outputs manifest.jsonl.
"""
import os
import json
import hashlib
from pathlib import Path
from datetime import datetime

VAULT_DIR = Path(os.environ.get("VAULT_DIR", "~/novae-xorpus")).expanduser()
OUTPUT_MANIFEST = VAULT_DIR / "manifest.jsonl"

def compute_sha256(filepath: Path) -> str:
    hasher = hashlib.sha256()
    with open(filepath, "rb") as f:
        while chunk := f.read(65536):
            hasher.update(chunk)
    return hasher.hexdigest()[:16]

def extract_metadata(filepath: Path) -> dict:
    title = filepath.stem
    description = ""
    tokens = set()

    try:
        with open(filepath, "r", encoding="utf-8", errors="ignore") as f:
            for line in f:
                stripped = line.strip()
                if not stripped or stripped.startswith("---"):
                    continue
                if stripped.startswith("# ") and title == filepath.stem:
                    title = stripped.lstrip("# ").strip()
                elif not description and not stripped.startswith("#"):
                    description = stripped[:200]
                if stripped.startswith("##"):
                    for word in stripped.lstrip("# ").lower().split():
                        if len(word) > 3 and word.isalnum():
                            tokens.add(word)
                if description and len(tokens) >= 10:
                    break
    except Exception as e:
```

```
        description = f"Extraction error: {e}"

    return {
        "title": title,
        "description": description or "No descriptive header located.",
        "tokens": list(tokens)[:15]
    }

def main():
    if not VAULT_DIR.exists():
        print(f"Directory {VAULT_DIR} not found.")
        return

    records = []
    supported_extensions = {".md", ".jsonl", ".py", ".sh", ".json", ".txt"}

    for root, _, files in os.walk(VAULT_DIR):
        if ".git" in root or ".obsidian" in root:
            continue
        for file in files:
            path = Path(root) / file
            if path.suffix.lower() in supported_extensions and path != OUTPUT_MANIFEST:
                meta = extract_metadata(path)
                rec = {
                    "record_id": f"REC_{compute_sha256(path)}_{path.stem.upper()[:24]}",
                    "file": path.name,
                    "path": str(path.relative_to(VAULT_DIR)),
                    "content_hash": compute_sha256(path),
                    "size_bytes": path.stat().st_size,
                    "modified": datetime.fromtimestamp(path.stat().st_mtime).isoformat(),
                    "title": meta["title"],
                    "summary": meta["description"],
                    "retrieval_tokens": meta["tokens"],
                    "entry_point": f"/read {str(path.relative_to(VAULT_DIR))}"
                }
                records.append(rec)

    with open(OUTPUT_MANIFEST, "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    print(f"Compiled {len(records)} entries into {OUTPUT_MANIFEST}")
```

```
if __name__ == "__main__":
    main()
```